4-DAY LEARNING SPRINT — PROFESSIONAL REPORT

My AI Learning Journey

A hands-on sprint from first prompt to a working, automated Gmail sending pipeline and a team-built Canvas art wall — covering AI tooling, Git & GitHub, cloud API integration, and team collaboration.

SHOAIBUL HOQUE

CONTENTS

1 Getting Started with AI & GitHub

DAY 1 · AUG 17, 2026

Day 1 was about getting my tools in order — setting up an AI coding partner, understanding what GitHub actually is, and shipping something real instead of just reading about it.

WHAT I DID	DETAILS
Set up Claude AI	Learned how to prompt it effectively — explaining what I wanted clearly, reviewing what it built, and asking follow-up questions instead of accepting the first answer.
GitHub fundamentals	Learned the basics of version control — what a repository is, why commits matter, and how a public repo lets anyone view or reuse the code.
Basketball Shooter	Built my first browser game from scratch using HTML, CSS, and JavaScript, with Claude helping me structure the game logic and scoring.
Manual GitHub upload	Uploaded the project to GitHub for the first time — creating a repository through the website and dragging the files in, rather than using Git commands (that came on Day 2).
AI-assisted workflow	Got comfortable describing the goal, letting Claude write the first draft, then testing it in the browser and asking for fixes.

AI Prompting	GitHub Basics	Game Development

2 Git Commands & Deployment

DAY 2 · AUG 18, 2026

After uploading files by hand on Day 1, Day 2 was about learning the proper way developers actually ship code — using Git from the terminal instead of the GitHub website. First, Git works in a real development workflow, tracking changes locally before they ever reach GitHub. Then I practiced the core Git loop used to ship code, end to end.

COMMAND	WHAT IT DOES
`git init`	Turned a normal folder into a Git repository.
`git add`	Staged the files I wanted to save a checkpoint of.
`git commit`	Saved that checkpoint with a message describing what changed.
`git push`	Sent the commit up to GitHub so it was live and visible to anyone.

WHAT ELSE I LEARNED	DETAILS
Updating code	Learned to edit, add, commit, push again — instead of re-uploading whole folders.
Shipping it live	Used this exact workflow to deploy the Basketball Shooter game and enable GitHub Pages, turning the repo into a live, playable website.

Basketball Shooter
The game I deployed live using Git on Day 2.
https://shoaibul0926.github.io/My-first-game/

PLAY THE GAME

Open this report's PDF and click the link above (or the button) to launch it in a browser.

3 Gmail API & Automation

DAY 3 · AUG 19, 2026

Day 3 moved from writing code to connecting real services — setting up Google's Gmail API and bringing Claude Pro into my terminal so I could automate a task end to end.

WHAT I DID	DETAILS
Google Cloud Platform	Connected my terminal to GCP to start building a real integration, not just a local script.
Auth & API integration	Learned the basics of authentication and API integration — why apps need permission before they can act on your behalf.
Send emails programmatically	Built a working pipeline to send emails straight from the terminal.

1. **Set up the Gmail API** in Google Cloud Console — created a project, enabled the Gmail API, and configured the OAuth consent screen.

2. **Restricted the access on purpose** — requested only a send-only permission, so the integration could send mail but never read my inbox.

3. **Completed the OAuth authorization** once through the browser consent screen, after which the terminal was trusted to send email without asking again each time.

4. **Installed the Claude Code CLI** and ran the login command to link my Claude Pro subscription to the terminal.

5. **From that point on** I could just type **claude** in the terminal and work with it directly — no browser tab needed — which is how the rest of this automation got built.

4 Team Collaboration & Canvas Animation

DAY 4 · AUG 20, 2026

Day 4 moved from working solo to working as part of a team — pushing to a shared repository without stepping on anyone else's work, and contributing one animated piece to a page made entirely of everyone's combined work.

WHAT I DID	DETAILS
Shared-repo Git collaboration	Pushed to a repository everyone on the team shared, with each person editing only their own file — a simple convention that avoided merge conflicts entirely.
GitHub Pages, revisited	Saw again how GitHub Pages turns a repo into a live website, this time watching a shared repo update as teammates pushed their own pieces.
HTML Canvas animation	Built a small animated scene using the HTML Canvas API and JavaScript for my piece of the group project.
Claude Code, hands-on	Used Claude Code to help generate and refine the animation, starting from a plain description of what I wanted and iterating from there.
Combined via iframes	My animation was joined with everyone else's into one shared page, each piece embedded as its own iframe on a single "art wall".

Group Art Wall
Everyone's Canvas animations combined into one page, including mine.
`https://mody-sahariar1.github.io/art-wall/`

VIEW THE WALL

Open this report's PDF and click the link above (or the button) to launch it in a browser.

Git Collaboration	GitHub Pages	Canvas Animation	Iframes	Claude Code